

**The University of Nottingham, UK
&
The University of Nottingham, Ningbo China**

SCHOOL OF COMPUTER SCIENCE

A LEVEL 1 MODULE, SPRING SEMESTER 2017-2018

FOUNDATIONS OF SOFTWARE ENGINEERING

Time allowed: **ONE HOUR AND THIRTY MINUTES**

Candidates may complete the front cover of their answer book and sign their desk card but must NOT write anything else until the start of the examination period is announced

Answer ALL questions

No calculators are permitted in this examination.

Dictionaries are not allowed with one exception. Those whose first language is not English may use a standard translation dictionary to translate between that language and English provided that neither language is the subject of this examination. Subject specific translation dictionaries are not permitted.

No electronic devices capable of storing and retrieving text, including electronic dictionaries, may be used.

DO NOT turn examination paper over until instructed to do so

ADDITIONAL MATERIAL: None.

INFORMATION FOR INVIGILATORS: Collect examination question papers at the end of the examination.

SECTION A: Software Engineering Activities

[Section A carries a total of 20 marks]

Question 1: Name and briefly describe the four fundamental activities common to all Software Engineering processes.

[4 marks]

[Bloom's Taxonomy: Knowledge]

1 mark for each of: Specification, Development, Validation, Evolution (or similar names) with correct explanations.

- all students should get this as its repeatedly referred to throughout the module

Question 2: Requirements and Specifications are often confused. Define both of them separately, and give an example of how one type of UML diagram can be used differently for each.

[4 marks]

[Bloom's Taxonomy: Knowledge, Comprehension]

1 mark for each definition, and 1 mark for each example.

1. Requirements is what the user wants to do
2. Specifications is how the system will be built
3. An Sequence diagram can be used to describe the tasks that a user wants to do with the system, at a higher level, for Requirements
4. The same diagram can be used to specify how classes call each others functions at the software level.

(points 3 and 4 might vary depending on which diagram they pick – but they should give a valid example of how the same diagram works at different levels of thinking)

Question 3: Prototyping is a popular way of exploring the design of a new system. Describe 3 common risks associated with prototyping.

[3 marks]

[Bloom's Taxonomy: Knowledge]

Any 3 of

1. Too much work done to make the decision needed
2. Prototype code makes it into development
3. Prototypes used instead of documentation

4. Prototypes approved by the wrong stakeholders

Question 4: Identify the type of testing that relates most to Requirements, and another type of testing that relates to High-Level Specifications. Describe two ways in which these testing phases are different.

[4 marks]

[Bloom's Taxonomy: Knowledge, Comprehension]

Essentially Book work

1. Requirements » Acceptance Testing
2. Specifications » Release Testing

More detailed comprehension (integrating lessons learned) - Along the lines of

3. Release Testing is internally in the company, Acceptance is done with the client
4. Release Testing fails goes back to the developers, Acceptance testing fails might go all the way back to the requirements phase.

Question 5: Describe how continuous integration can be used to make sure software is ready for deployment on different platforms.

[5 marks]

[Bloom's Taxonomy: Comprehension, Application]

As a more open short-essay question, we expect the student to a) define continuous integration (1 mark), and b) describe a process (1 mark) that ties together 3 aspects (1 mark each).

Continuous Integration – where all code is submitted to a project is integrated directly into product mainline, so that software is always working.

Process – automated testing of code to make sure it is ok, code is auto built with configuration scripts for each platform, and each compiled version is automatically deployed to test machines.

1. Automated testing – CI triggers automatic tests to make sure code contributions pass the test.
2. Configuration Scripts – These describe how code should be compiled, with which libraries, for a platform. Have different configuration scripts for each platform (or old version of platform).
3. Deployment to test machines – automatic deployment onto real or virtual test machines to check that they deploy correctly – logs watched by external process.

SECTION B: Project Management & Risks

[Section B carries a total of 15 marks]

Question 6: Project Risks and Software Risks are two types of Software Engineering Risks that project managers are concerned with. Define **Software Risks**, and describe how considering them should affect two key stages of the Software Engineering Process.

[3 marks]**[Bloom's Taxonomy: Knowledge, Comprehension]**

1. Software Risks are those that relate to security or data protection type issues.
2. This should lead to additional specifications, including SHOULD NOTs (may refer back to requirements)
3. This should lead to additional testing

Question 7: The husband of your most experienced senior developer has been diagnosed with a long-term illness, and they may need to take leave several times during the project. Everyone else on the team are relatively junior developers. This is a **Project Risk** that can be mitigated with team organisation strategies.

- (i) Describe two ways that Paired Programming can help the team mitigate this risk.

[2 marks]**[Bloom's Taxonomy: Knowledge, Comprehension]**

Should be able to sample from the knowledge of Paired Programming, for which are relevant here.

1. Junior developers can learn from the senior developer
2. Someone else will always know how the code works if the senior engineer is away.

- (ii) Decide whether this senior developer should be assigned to Critical Path tasks or not. Give two reasons for your decision, and one counter argument.

[4 marks]**[Bloom's Taxonomy: Comprehension, Application]**

Don't mind which way they go on this one. Expect 1 mark for clear decision. 2 marks for clear reasons. 1 mark for clear counter argument.

Reasons for include: these are the most important to keep on time, or they can teach junior developers on more critical work.

Reasons against: its risky to have someone who might be away on the critical path tasks, or you want to always have two people on the more critical path.

Question 8: Experienced companies often employ a Software Quality Assurance Team to monitor Software Quality across all projects.

- (i) Explain why depending on the testing team to manage Software Quality could create a Project Risk.

[2 marks]

[Bloom's Taxonomy: Knowledge, Comprehension]

This is an interpretation of knowledge from several points in the course. Two points along the lines of:

1. Finding a problem at testing time means its too late to fix a problem
2. Fixing problems after testing can be expensive
3. They cant easily tell if requirements were high quality in the first place.

- (ii) Identify four other Software Quality Assurance activities that can be used at other stages of the Software Engineering Process to improve the Software Quality.

[4 marks]

[Bloom's Taxonomy: Comprehension, Application]

Requires them to take a principle taught (about finding ways to improve process) and think actually about where they could do that (not taught). Four points like

- 1) Improving requirements validation activities
- 2) Organising inspections of e.g. specification documents
- 3) Reviewing prototypes with clients
- 4) Reviewing code conventions or naming policies
- 5) Improving testing protocols (e.g. separating test teams from developers at release levels)
- 6) Defining documentation standards

SECTION C: Agile Methodologies

[Section C carries a total of 15 marks]**Read the following scenario and answer the questions below.**

You work as a developer in a large software company called InTheBank Apps, in the open-plan office, as part of a development team. Your company also has a strong requirements engineering team, and a testing and quality assurance team. The company has already gained a reputation for building high quality financial software for banks, conforming to the various legal and financial standards. You recently used traditional Software Engineering methods to rebuild the core financial software for The Major Bank of Bankland.

The Major Bank of Bankland has approached your company again, but to do something a little different this time: create an app for customers to track their financial spending for different types of things, like Bills, Shopping, Socialising, etc. It's a very vague idea so far. The app, however, is an extension of the previous software and so the project will still handle important financial data, but the users are clients of the bank, not staff at the bank. Because it's a new idea, the bank wants to explore lots of different options, but has identified a working group of potential users willing to come to a monthly discussion evening.

Question 9: Identify and briefly describe the four original values of the Agile Manifesto.

[4 marks]

[Bloom's Taxonomy: Knowledge]

Should list (or similar wording):

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

all students should get this. Students may confuse this with the 5 principles boiled down from the 12 principles by Sommerville. In which case they might say maintain simplicity, which should not be accepted a mark.

Question 10: Explain how User Stories, as an agile form of requirements documentation, can be used to represent different stages of the Software Engineering process.

[4 marks]

[Bloom's Taxonomy: Comprehension]

Not explicitly taught – requires them to think about different SE phases and explain how User Stories can be used there. Expecting a sample from the following (but would except reasonable different example).

1. Requirements Definition – to capture different uses of the system
2. System Design – to examine designs from different actors
3. Implementation planning – selection of user stories for current dev sprint etc
4. Testing – used to run release tests
5. Documentation – as part of final documentation

Question 11: You and the rest of the project team at InTheBank Apps are considering whether, for this project, to use a traditional plan-based Software Engineering process or a more Agile Software Engineering process to develop the idea.

- (i) Describe two reasons, from the scenario, for adopting each approach.

[4 marks]

- (ii) Based upon these reasons, choose the approach you believe is most appropriate for the primary approach for this project. Complement this decision with two ways in which you will integrate the alternative approach.

[3 marks]

[Bloom's Taxonomy: Application]

Requires them to interpret the scenario.

(i)

- a. Reasons for plan: legal standards, high risk data, prior experience, different teams exist already
- b. Reasons for agile: no clear idea yet, want to do lots of design, have access to user groups

(ii)

- Doesn't matter so much which they pick (1 mark) – but should say something like
- a. We should do formal – but consult user group with each deliverable, and use agile development
 - b. We should do agile – but include legal standards in testing phase, and begin with a more detailed requirements phase.